

Data Analysis with Python

Lesson 7: Introduction to NumPy for Numerical Work

Introduction to NumPy for Numerical Work - Study Notes

Key Concepts

1. **NumPy arrays (`ndarray`)** – homogeneous, multi dimensional containers that replace Python lists for numerical tasks.
2. **Vectorized operations** – applying arithmetic or logical operations element wise without explicit Python loops, yielding huge speedups.
3. **Broadcasting** – automatically expanding arrays of different shapes so they can participate in element wise operations.
4. **Array creation and manipulation** – functions such as `np.arange`, `np.linspace`, `np.zeros`, `np.ones`, `np.reshape`, and indexing/slicing.
5. **Memory layout and performance** – contiguous vs. non contiguous memory, `C`` vs. `F`` order, and the importance of avoiding Python loops.

Summary

NumPy is the foundational library for numerical computing in Python. Its core data structure, the `ndarray`, stores elements of a single data type in a contiguous block of memory, enabling efficient storage and fast vectorized operations. By leveraging NumPy's broadcasting rules, you can perform operations on arrays of different shapes without explicit loops, which dramatically speeds up code and reduces memory overhead. Mastering array creation, slicing, and reshaping is essential for transforming raw data into the shapes required by algorithms. Understanding the underlying memory layout helps you write performance critical code and avoid common pitfalls such as unintentionally creating copies.

Important Points

- **Importing**: `import numpy as np` is the conventional alias.
- **Array creation**:

```
```python
a = np.array([1, 2, 3]) # 1 D array
b = np.arange(0, 10, 2) # 0,2,4,6,8
c = np.linspace(0, 1, 5) # 5 evenly spaced numbers
```
```

- **Data types**: Use `dtype` argument to control precision (`float32`, `int64`, etc.).
- **Vectorized ops**: `a + b`, `np.sin(a)`, `a * 2`.
- **Broadcasting**: A 1 D array of shape `(n,)` can be added to a 2 D array of shape `(m, n)`; NumPy automatically replicates the 1 D array along the new axis.
- **Indexing**:
 - Basic slicing: `a[1:5]`

- Boolean indexing: `a[a > 3]`
- Fancy indexing: `a[[0, 2, 4]]`
 - **Reshaping**: `a.reshape((2, 3))` – must preserve total number of elements.
 - **Memory layout**: `a.flags` shows `C_CONTIGUOUS` or `F_CONTIGUOUS`. Use `np.ascontiguousarray` or `np.asfortranarray` to enforce layout.
 - **Performance**: Avoid Python loops; use `np.vectorize` sparingly (it's just a convenience wrapper, not a speed up).
 - **In place operations**: Use `+=`, `*=`, etc., to modify arrays without allocating new memory.

Examples

1. Vectorized arithmetic and broadcasting

```
```python
```

```
import numpy as np
```

#### Create a 2 D array (3 rows, 4 columns)

```
A = np.arange(12).reshape(3, 4)
```

```
print("A:\n", A)
```

#### Add a 1 D array to each row (broadcasting)

```
row_offset = np.array([10, 20, 30])
```

```
B = A + row_offset[:, None] # shape (3,1) broadcasts to (3,4)
```

```
print("\nB (A + row_offset):\n", B)
```

```
```
```

Explanation

- `row_offset[:, None]` turns the shape `(3,)` into `(3,1)`.
- Broadcasting expands it to match the shape of `A`.
- Result: each row of `A` is increased by a different offset.

2. Masked operations and statistical functions

```
```python
```

#### Random data

```
np.random.seed(0)
```

```
data = np.random.randn(5, 5)
```

#### Mask: keep only values > 0

```
mask = data > 0
```

```
positive_values = data[mask]
```

```
print("Positive values:", positive_values)
```

#### Compute mean and std of positive values

```
mean_pos = positive_values.mean()
std_pos = positive_values.std()
print(f"Mean: {mean_pos:.3f}, Std: {std_pos:.3f}")
...
```

#### **\*\*Explanation\*\***

- Boolean masking extracts elements that satisfy a condition.
- Statistical functions (``mean``, ``std``) work directly on the masked array, avoiding explicit loops.

---

## **Quick Review**

1. What is the primary data structure used in NumPy for numerical work, and why is it efficient?
2. Explain how broadcasting works when adding a 1-D array to a 2-D array.
3. How can you reshape a 1-D array of length 12 into a 3×4 matrix?
4. What is the difference between ``np.array([1, 2, 3])`` and ``np.arange(1, 4)``?
5. Why should you avoid Python loops when performing element-wise operations on large arrays?

---

## **Further Reading**

- **\*\*Linear algebra with NumPy\*\*** – ``np.linalg`` module (eigenvalues, SVD).
- **\*\*Random number generation\*\*** – ``np.random`` submodule and reproducibility.
- **\*\*Advanced indexing\*\*** – integer, boolean, and slice indexing tricks.
- **\*\*Memory layout and performance tuning\*\*** – ``np.dot``, ``np.einsum``, and C extensions.
- **\*\*Integration with pandas\*\*** – converting between NumPy arrays and pandas DataFrames/Series.
- **\*\*NumPy's ufuncs\*\*** – universal functions and how they enable vectorized math.

